

From Vectors Back to Words

Logits, softmax, and choosing the next token · video 2:14

After the last block, every token is still a vector. This lesson turns the last one back into a word: the **unembedding matrix** gives **one score (logit) per token**, **softmax** turns the scores into probabilities, and a **sampling rule** (greedy, temperature, top-k, top-p) picks the next token. That closes the loop from episode 1.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 10).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. From the last vector to logits	▶ 0:08-0:39	4	☐
2. Each logit is a dot product	▶ 0:40-0:48	4	☐
3. Softmax: scores to probabilities	▶ 0:49-1:02	4	☐
4. Greedy picking and temperature	▶ 1:02-1:26	4	☐
5. Top-k and top-p	▶ 1:26-1:39	4	☐
6. One sentence, many lessons	▶ 1:40-1:52	3	☐
7. Choosing the next token, in code	▶ 1:52-2:12	3	☐

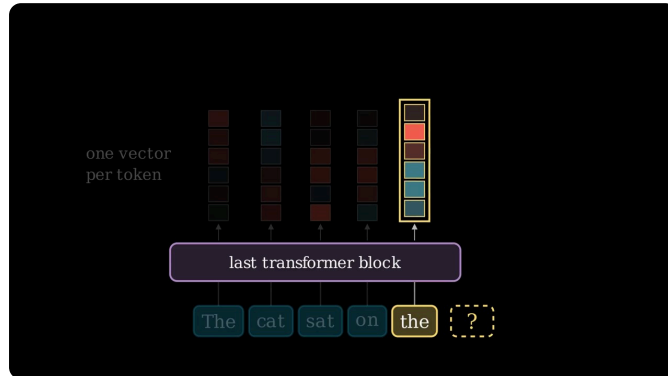
Before you start: this lesson builds on episode 1 (predict, pick, append, repeat), episode 3 (the embedding matrix) and episode 9 (the stack of blocks). For the maths, Foundations F02 (The Dot Product), F03 (Matrices), F05 (Exponentials and Logarithms), F06 (Probability and Sampling) and F07 (Softmax, Properly) cover everything used here.

CONCEPT 1 OF 7

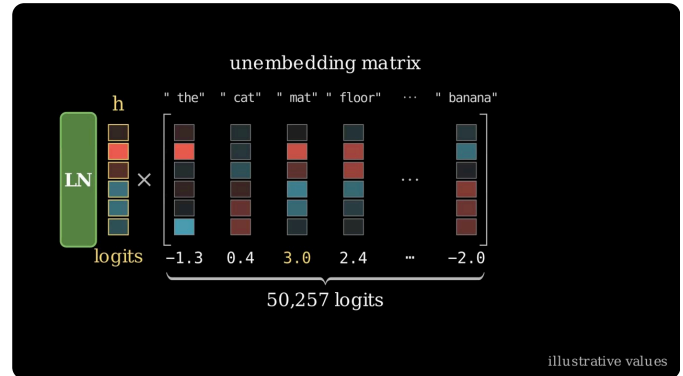
From the last vector to logits

▶ Video 0:08-0:39

After the last transformer block there is one vector per token. To predict what comes after “The cat sat on the”, we only need **the last one**: the vector sitting on the word *the*.



▶ 0:18 Only the last vector, on “the”, is needed.



▶ 0:34 One column, and one logit, per vocabulary token.

It goes through **one last layer norm**, then is multiplied by the **unembedding matrix**, which has **one column for every token** in the vocabulary. The result is one score per token, **50,257 numbers** in GPT-2, called **logits**. In GPT-2 this matrix is the **embedding matrix again**, reused (“tied weights”).

KEY IDEA

Last vector → final layer norm → × unembedding matrix → **one logit per vocabulary token**.

Check 1: From the last vector to logits

Answer all 4 questions before looking at the answers. Pass mark: all correct.

1.1 MULTIPLE CHOICE

Which vector is used to predict the word after “The cat sat on the”?

- A. The vector on the first word, *The*
- B. The average of all five vectors
- C. The vector on the last token, *the*
- D. The vector on *cat*, the most important word

1.2 CALCULATION

GPT-2 small has vectors of 768 numbers and a vocabulary of 50,257 tokens. (a) How many logits does one prediction produce? (b) How many numbers are in the unembedding matrix?

1.3 TRUE OR FALSE

“In GPT-2, the unembedding matrix is learned separately from the embedding matrix.”

True ☐ False ☐

1.4 PUT IN ORDER

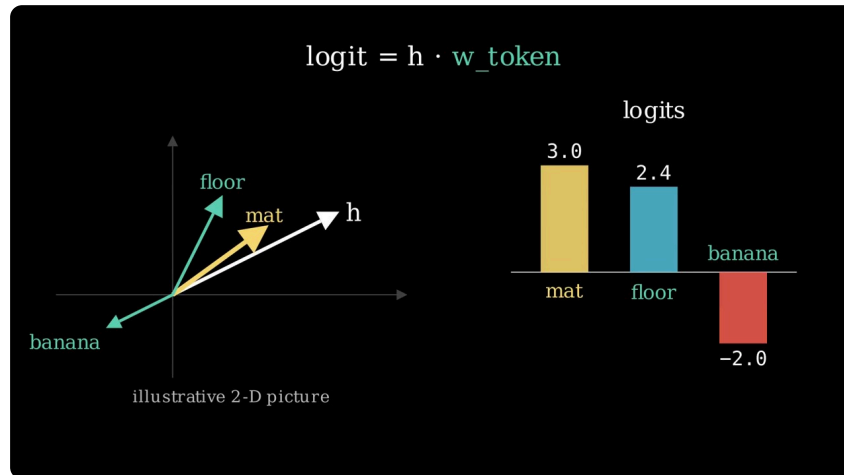
Put in order: *unembedding matrix* · *last transformer block* · *logits* · *final layer norm*.

CONCEPT 2 OF 7

Each logit is a dot product

▶ Video 0:40–0:48

Multiplying by the matrix means taking a **dot product** with each token's column: **logit** = $\mathbf{h} \cdot \mathbf{w}_{\text{token}}$. It measures how well the final vector $\mathbf{h}$ **lines up** with that token's direction. The better the match, the higher the score.



▶ 0:48 In a 2-D picture: *mat* lines up best with $\mathbf{h}$ and gets the highest score; *banana* points away and gets a negative one.

In the video's 2-D picture, $\mathbf{h} = (2, 1)$. *mat* = $(1.1, 0.8)$ scores $2 \cdot 1.1 + 1 \cdot 0.8 = \mathbf{3.0}$, *floor* = $(0.6, 1.2)$ scores **2.4**, and *banana* = $(-0.8, -0.4)$, pointing the other way, scores **-2.0**.

KEY IDEA

A logit is a dot product: **lined up** → **high**, at right angles → 0, **opposite** → **negative**.

Check 2: Each logit is a dot product

Answer all 4 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

With $\mathbf{h} = (2, 1)$, compute the logits for *sofa*, $\mathbf{w} = (0.5, 0.9)$, and for *bed*, $\mathbf{w} = (0.4, 0.8)$.

2.2 SHORT ANSWER

A new token *rug* has $\mathbf{w} = (1.2, 0.7)$. Would it beat *mat* (3.0)?

2.3 MULTIPLE CHOICE

With $\mathbf{h} = (2, 1)$, which token direction gives the most negative logit?

- A. $(2, 1)$
- B. $(1, -2)$
- C. $(-2, -1)$
- D. $(0, 0)$

2.4 TRUE OR FALSE

"A logit is always a number between 0 and 1."

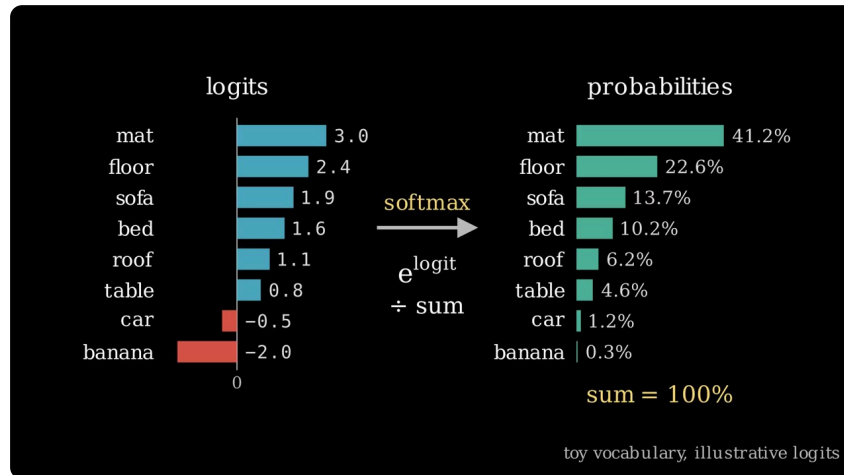
True ☐ False ☐

CONCEPT 3 OF 7

Softmax: scores to probabilities

▶ Video 0:49–1:02

Logits can be any number. **Softmax** turns them into probabilities: **exponentiate** each one (e^{logit} is always positive), then **divide by the total**. Now every token has a probability, and they add up to 1.



▶ 1:02 The toy vocabulary: logits on the left, probabilities after softmax on the right. They add up to 100 %.

In the toy vocabulary the eight values e^{logit} add up to about 48.72, and $e^{3.0} \approx 20.09$, so *mat* gets $20.09 / 48.72 \approx 41.2\%$. Even *banana* (logit -2.0) gets a small positive share: 0.3 %.

KEY IDEA

softmax: $p = e^{\text{logit}} / (\text{sum of all } e^{\text{logit}})$. A bigger logit gives a bigger probability; every probability is positive; they add up to 1.

Check 3: Softmax: scores to probabilities

Answer all 4 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

Compute the softmax of the logits $[2, 1, 0]$, as percentages ($e^2 \approx 7.39$, $e^1 \approx 2.72$, $e^0 = 1$).

3.2 CALCULATION

In the toy vocabulary, $e^{2.4} \approx 11.02$. Using the total 48.72, what is *floor*'s probability?

3.3 SHORT ANSWER

You add 10 to every logit. What happens to the probabilities?

3.4 TRUE OR FALSE

"A token with a negative logit gets a negative probability."

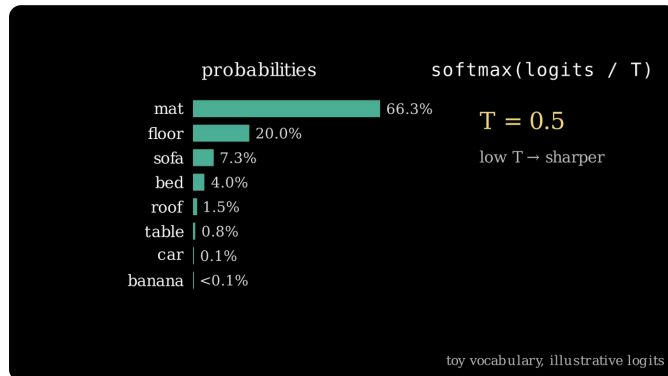
True ☐ False ☐

CONCEPT 4 OF 7

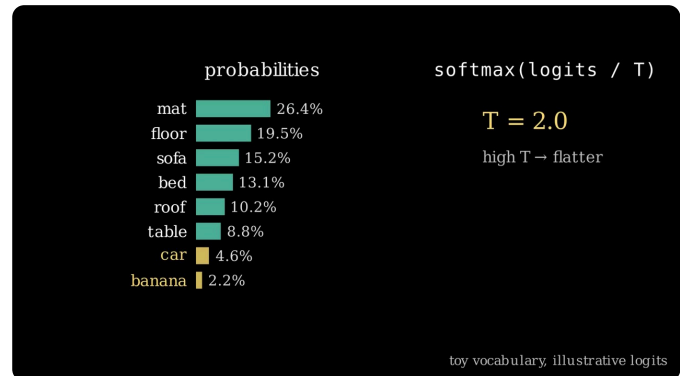
Greedy picking and temperature

▶ Video 1:02-1:26

The simplest choice is **greedy**: always take the most likely token. But that tends to be **repetitive and dull**. Instead we **sample**, and control how adventurous that is with the **temperature** T : divide the logits by T before the softmax, $\text{softmax}(\text{logits} / T)$.



▶ 1:21 Low temperature ($T = 0.5$): sharper, *mat* takes 66 %.



▶ 1:25 High temperature ($T = 2$): flatter, rare words get a chance.

A **low** T **sharpens** the distribution toward the top choice (*mat*: 41.2 % $\rightarrow$ 66.3 % at $T = 0.5$). A **high** T **flattens** it, so rarer words get a chance (*banana*: 0.3 % $\rightarrow$ 2.2 % at $T = 2$).

KEY IDEA

$T < 1$: sharper, closer to greedy. $T > 1$: flatter, more variety. $T = 1$: the model's own probabilities.

Check 4: Greedy picking and temperature

Answer all 4 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

At $T = 0.5$, what do the logits of *mat* (3.0) and *floor* (2.4) become? How does the gap between them change?

4.2 CALCULATION

At $T = 1$, *mat* (41.2 %) is about 1.8 times as likely as *floor* (22.6 %). Using the two charts above, how many times as likely is it at $T = 0.5$, and at $T = 2$?

4.3 TRUE OR FALSE

"A very low temperature, such as $T = 0.1$, makes sampling behave almost like greedy picking."

True ☐ False ☐

4.4 SHORT ANSWER

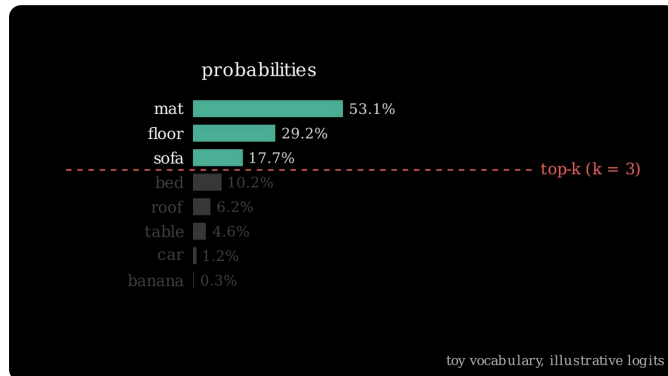
You want a story generator to be more surprising. Should you raise or lower the temperature?

CONCEPT 5 OF 7

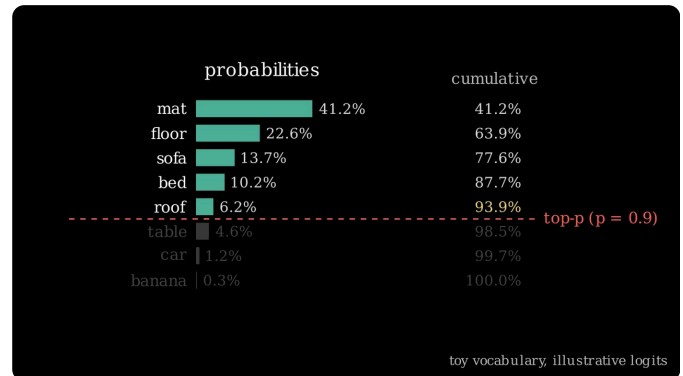
Top-k and top-p

▶ Video 1:26–1:39

Two more tricks keep sampling sensible by cutting unlikely tokens before we sample. **Top-k** keeps only the **k most likely** tokens. **Top-p** keeps the **smallest set** of top tokens whose probabilities **add up to p** (at least p), say 90 %. The kept probabilities are rescaled to add up to 1, then we sample.



▶ 1:31 Top-k ($k = 3$): keep the three most likely tokens and rescale.



▶ 1:37 Top-p ($p = 0.9$): keep tokens until the running total reaches 90 %.

With $k = 3$, *mat*, *floor* and *sofa* become 53.1 %, 29.2 % and 17.7 %. With $p = 0.9$, the running total first reaches 90 % at *roof* (93.9 %), so five tokens stay.

KEY IDEA

Top-k keeps a **fixed number** of tokens; top-p keeps **as many as needed** to cover p. Cut, rescale, sample.

Check 5: Top-k and top-p

Answer all 4 questions before looking at the answers. Pass mark: all correct.

5.1 CALCULATION

Use top-k with $k = 2$ on the $T = 1$ chart (*mat* 41.2 %, *floor* 22.6 %). What are the new probabilities?

5.2 CALCULATION

The running totals at $T = 1$ are 41.2, 63.9, 77.6, 87.7, 93.9 and 98.5 %. How many tokens does top-p keep for $p = 0.5$? And for $p = 0.8$?

5.3 SHORT ANSWER

At $T = 2$ the chart is flatter: the running totals are 26.4, 45.9, 61.2, 74.3, 84.5, 93.2 and 97.8 %. How many tokens does top-p ($p = 0.9$) keep now? And top-k ($k = 3$)?

5.4 TRUE OR FALSE

“After the cut, the kept probabilities are rescaled so they add up to 1 again.”

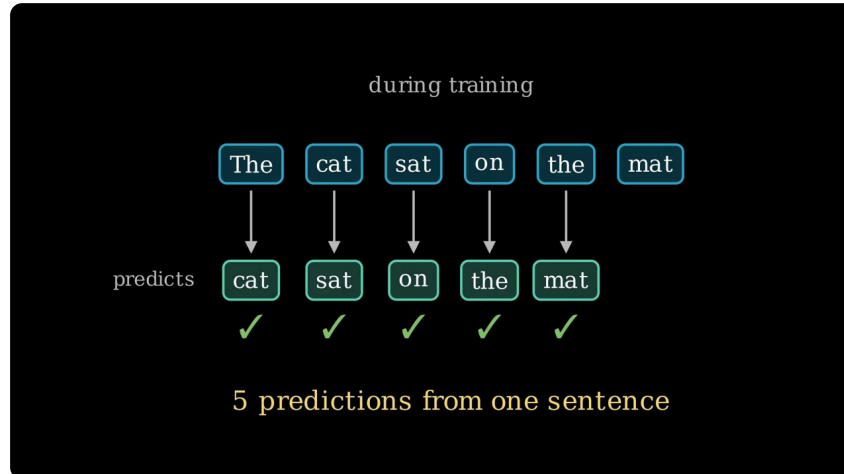
True ☐ False ☐

CONCEPT 6 OF 7

One sentence, many lessons

▶ Video 1:40–1:52

During **training** the model does not predict just one next token. **Every position** predicts its own next token **at the same time**, and each prediction is checked against the **real** next token in the text.



▶ 1:51 During training, every position predicts its own next token, and each prediction is checked against the real one.

So one sentence gives the model **many lessons at once**: “The cat sat on the mat” (6 tokens) gives 5 predictions to check. Episode 11 shows how the checking turns into learning.

KEY IDEA

In training, a text of n tokens gives $n - 1$ next-token predictions at the same time, each checked against the real next token.

Check 6: One sentence, many lessons

Answer all 3 questions before looking at the answers. Pass mark: all correct.

6.1 CALCULATION

A training text has 10 tokens. How many next-token predictions can be checked?

.....

6.2 SHORT ANSWER

In “The cat sat on the mat”, what should the position on *sat* predict? And the position on the second *the*?

.....

6.3 TRUE OR FALSE

“During training, the model generates the sentence one word at a time, and only the final word is checked.”

True ☐ False ☐

CONCEPT 7 OF 7

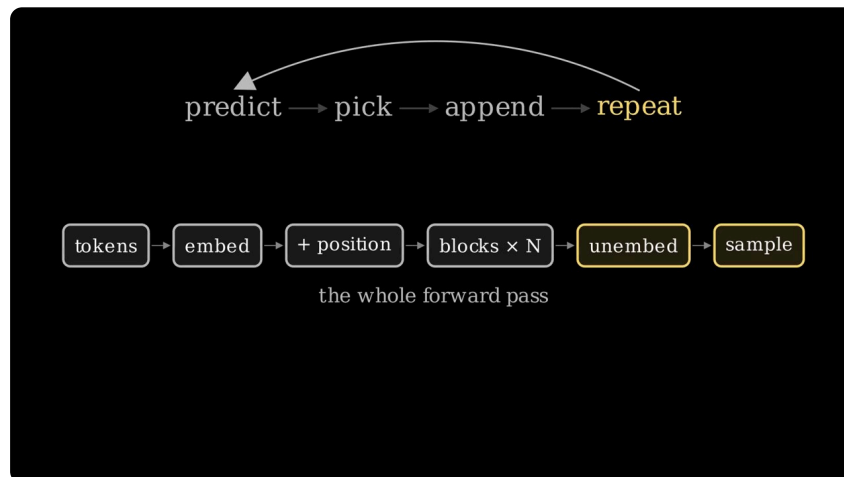
Choosing the next token, in code

▶ Video 1:52-2:12

In code: compute the logits and divide by the temperature, keep the top k, softmax, and sample one token.

```
def next_token(h, W_unembed, temperature=0.8, top_k=50):  
    logits = h @ W_unembed / temperature    # a score per token  
    top = np.argsort(logits)[-top_k:]        # keep the k best  
    p = np.exp(logits[top] - logits[top].max())  
    p /= p.sum()                             # softmax  
    return np.random.choice(top, p=p)        # sample one
```

That closes the loop from episode 1: **predict, pick, append, repeat**. We have now seen the **whole forward pass**. But all those matrices started out as **random noise**. How do they learn? Episode 11: training.



▶ 2:05 The whole forward pass, from tokens to a sampled next token.

KEY IDEA

`next_token` : logits / T → keep the top k → softmax → sample. Put it inside the episode 1 loop and the model writes text.

Check 7: Choosing the next token, in code

Answer all 3 questions before looking at the answers. Pass mark: all correct.

7.1 PUT IN ORDER

Put the whole forward pass in order: *sample · blocks × N · tokens · unembed · + position · embed*.

7.2 MULTIPLE CHOICE

The line `p = np.exp(logits[top] - logits[top].max())` subtracts the largest logit before exponentiating. Why is that safe?

- A. It changes the probabilities in favour of rarer tokens
- B. Subtracting the same number from every logit does not change the softmax, and it keeps e^x from getting huge
- C. It is a bug, fixed by the next line
- D. It is how the temperature is applied

7.3 HANDS-ON CODE

Try it yourself. Here is a version of `next_token` that takes the logits directly (and also returns `top` and `p`), with the toy vocabulary. (a) What does the `print` show? (b) Call `next_token(logits)` 10,000 times and count the words. About what share is *mat*? Does *banana* ever come out? (c) Try `temperature=0.1`: what is `p` now, and which strategy from concept 4 does this approach?

```
import numpy as np

WORDS = ["mat", "floor", "sofa", "bed", "roof", "table", "car", "banana"]
logits = np.array([3.0, 2.4, 1.9, 1.6, 1.1, 0.8, -0.5, -2.0])

def next_token(logits, temperature=1.0, top_k=3):
    logits = logits / temperature
    top = np.argsort(logits)[-top_k:]          # keep the k best
    p = np.exp(logits[top] - logits[top].max())
    p /= p.sum()                             # softmax
    return np.random.choice(top, p=p), top, p

tok, top, p = next_token(logits)
print(top, np.round(p, 3))
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. From the last vector to logits · rewatch ▶ 0:08-0:39

1.1 C. The prediction for the next token is read from the last position's vector.

1.2 (a) 50,257. (b) $768 \times 50,257 = 38,597,376$. One column of 768 numbers per vocabulary token. With tied weights, these are the same numbers as the embedding matrix.

1.3 False. It is the embedding matrix again (transposed), reused: tied weights.

1.4 last transformer block → final layer norm → unembedding matrix → logits. The last vector is normalized once more, then multiplied by the matrix to give the scores.

2. Each logit is a dot product · rewatch ▶ 0:40-0:48

2.1 1.9 and 1.6. $2 \cdot 0.5 + 1 \cdot 0.9 = 1.9$ and $2 \cdot 0.4 + 1 \cdot 0.8 = 1.6$: the values of the toy vocabulary in concept 3.

2.2 Yes: its logit is 3.1. $2 \cdot 1.2 + 1 \cdot 0.7 = 3.1 > 3.0$: it lines up with h even better.

2.3 C. $(-2, -1)$ points exactly opposite to h: -5 . $(1, -2)$ is at right angles (0), $(0, 0)$ gives 0, and $(2, 1)$ gives $+5$.

2.4 False. Logits can be any number, positive or negative (*banana*: -2.0). Softmax turns them into probabilities.

3. Softmax: scores to probabilities · rewatch ▶ 0:49-1:02

3.1 $\approx 66.5\%$, 24.5% , 9.0% . The total is 11.11: $7.39 / 11.11$, $2.72 / 11.11$ and $1 / 11.11$.

3.2 $\approx 22.6\%$. $11.02 / 48.72 \approx 0.226$, as in the chart.

3.3 Nothing: they stay exactly the same. $e^{\text{logit} + 10} = e^{10} \cdot e^{\text{logit}}$. Every term and the total get the same factor, which cancels in the division: only the differences between logits matter.

3.4 False. e^x is positive for every x: *banana*'s -2.0 becomes 0.3% .

4. Greedy picking and temperature · rewatch ▶ 1:02-1:26

4.1 6.0 and 4.8: the gap doubles from 0.6 to 1.2. Dividing by 0.5 doubles every logit, so the differences grow and the top choice pulls ahead.

4.2 About 3.3 and about 1.35. $66.3 / 20.0 \approx 3.3$ and $26.4 / 19.5 \approx 1.35$. (Exactly: $e^{1.2}$ and $e^{0.3}$, the gap divided by T.)

4.3 True. At $T = 0.1$, *mat* gets 99.75 %, so it is picked almost every time.

4.4 Raise it ($T > 1$). A higher T flattens the distribution, so less likely words are picked more often. A lower T (or greedy) makes the output more predictable.

5. Top-k and top-p · rewatch ► 1:26-1:39

5.1 $\approx 64.6\%$ and 35.4% . $41.2 / 63.8$ and $22.6 / 63.8$: the kept tokens are rescaled to add up to 100 %.

5.2 **2 and 4**. The smallest set that reaches p: $63.9\% \geq 50\%$ after two tokens, $87.7\% \geq 80\%$ after four.

5.3 **Six; still three**. Top-p adapts to the shape of the distribution (flatter means more tokens); top-k always keeps exactly k.

5.4 **True**. $53.1 + 29.2 + 17.7 = 100\%$: each kept token grows in proportion.

6. One sentence, many lessons · rewatch ► 1:40-1:52

6.1 **9**. Every position except the last has a real next token to be checked against.

6.2 **on; mat**. Each position's target is simply the next token in the text.

6.3 **False**. All positions predict at the same time, and every prediction is checked.

7. Choosing the next token, in code · rewatch ► 1:52-2:12

7.1 **tokens → embed → + position → blocks × N → unembed → sample**. Episodes $2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 9 \rightarrow 10$.

7.2 **B**. Concept 3, exercise 3.3: only differences between logits matter. The largest value becomes $e^0 = 1$, so nothing overflows.

7.3 **(a) [2 1 0] [0.177 0.292 0.531] · (b) about 53 % mat; banana never · (c) $p \approx [0. 0.002 0.998]$: almost greedy** (a) `argsort` sorts from smallest to largest, so the last three indices are sofa (2), floor (1) and mat (0); p is the top-3 softmax, the $17.7 / 29.2 / 53.1\%$ of concept 5. (b) Sampling follows p, so mat comes out about 5,300 times in 10,000; banana is cut by top-k, so its chance is exactly 0. For example: `words = [WORDS[next_token(logits)[0]] for _ in range(10000)]`, then `words.count("mat")`. (c) Dividing by 0.1 turns the gaps 0.6 and 1.1 into 6 and 11, so mat takes 99.8 %: nearly greedy.